

Experiential Learning of Runtime Monitoring Using Pachinko

Miles Scharff 

Barnard College, Columbia University, New York City

Maria Chemodanova 

Barnard College, Columbia University, New York City

Mark Santolucito 

Barnard College, Columbia University, New York City

Abstract

We present documentation of a classroom assignment that teaches runtime monitoring through a creative embedded systems build: an interactive Pachinko game. The assignment centers on a dual-core ESP32 workflow in which students write RTLola specifications for monitors, compile these monitors to C, and deploy them alongside sensor and actuator control logic. Pachinko game events are logged in real time and used to trigger sound, animation, and motor behavior according to formal temporal logic specifications.

This work showcases how formal methods can be taught in a hands-on, project-based setting for learners in a creative and classroom-scale setting. We also discuss portability: the assignment template, hardware stack, code base, and assessment approach are designed and documented to be replicated in other embedded systems, creative computing, or makerspace-style courses. This assignment was given to the students of Creative Embedded Systems (COMS3930) at Barnard College.

2012 ACM Subject Classification Software and its engineering → Formal methods

Keywords and phrases Runtime Monitoring, CS Education

Digital Object Identifier 10.5281/zenodo.21223137

Supplementary Material <https://github.com/Barnard-PL-Labs/Runtime-Monitoring-Pachinko>

Funding Miles Scharff: [NSF#2525332]

Mark Santolucito: [NSF#2525332]

1 Introduction

Runtime monitoring is a formal verification technique used for real time critical cyber-physical systems such as drones or aircraft. It is often implemented as a separate module, the “monitor”, which receives data streams from other parts of the system and runs these streams through the monitor to guarantee that the data adheres to some formal correctness specification. The goal is to have a safety net so that if some subsystem fails, the runtime monitor is able to be a last resort system to catch such errors and raise an alarm to an appropriate handler. Runtime monitoring is especially useful in large, complex systems, such as flight systems or other systems considered mission-critical, that are worked on by many people or teams. As described in a NASA report on their runtime monitoring tool Copilot, they estimate the functional correctness of 10,000 lines of code would take a team of 20 engineers years to achieve [6], hence runtime monitoring is a necessary solution for mission-critical systems. This context of large teams working on complex mission-critical systems is often what makes runtime monitoring so difficult to motivate in an educational setting. The reality of classroom appropriate project scoping means the motivation for runtime monitoring is difficult to replicate.





■ **Figure 1** Student pachinko board projects from Spring 2026 in COMS3930 at Barnard College, Columbia University.

44 We developed the Pachinko assignment to overcome these educational barriers and create
 45 an engaging and relevant introduction to runtime monitoring. Pachinko is a Japanese game
 46 involving steel balls falling down on to nails that determine their path. The goal of the player
 47 is to try and get as many balls into specific locations at the bottom of the board as possible.
 48 The assignment centers around an ESP32¹ at the bottom of the Pachinko board that counts
 49 how many times a ball will roll into various positions. This ESP32 is running a C-compiled
 50 version of RTLola [2, 4, 1], a runtime monitoring tool, and pipes the behavior of these balls
 51 through formal specification. When a runtime monitor condition is triggered, the ESP32
 52 sends messages to other the ESP32's on the board that control stepper motors to affect ball
 53 behavior. These messages are sent wirelessly through the ESP-NOW protocol [3].

54 In order for the students to complete the assignment successfully, they must write a
 55 formal specification in RTLola that triggers behaviors in their Pachinko game that satisfy
 56 their chosen artistic themes. In this context, system complexity is validated through artistic
 57 drive and simulated through the inherent noisy unpredictability of ball behavior in Pachinko.
 58 There is also a need to react to the ball distributions in real time. Hence, runtime monitoring
 59 and formal specifications feel well-motivated to use in this context.

60 Our goal for this first deployment of this project was to see how well students were able
 61 to write RTLola formal specifications to guide the desired behavior of their Pachinko game.

¹ The ESP32, developed by Espressif (<https://www.espressif.com/en/products/socs/esp32>), is an Arduino-like device that is often used in introductory embedded systems courses.

2 Background and Related Work

2.1 Creative Embedded Systems Course

The assignment was developed in the context of a broader curricular effort to integrate formal methods into creative embedded-systems education at Barnard College and Columbia University. That effort targets students building interactive media systems (e.g., sound, light, and installation pieces) on microcontroller platforms, where behavior is reactive, concurrent, and often distributed across multiple boards.

A key motivation is that creative-technology students increasingly use AI-assisted programming to build systems whose complexity can outpace their ability to reason about correctness. The curriculum therefore introduces lightweight formal methods directly in the toolchains students already use, including runtime verification [5, 7] with RTLola.

This pedagogical framing treats formal specification as practical design documentation for interactive behavior rather than an abstract prerequisite. Prior course experience in creative embedded systems also provides natural settings for properties involving message passing, timing, and sensor-driven state changes, making the course a strong environment for introducing verification concepts to new communities.

3 Pachinko Assignment

Students are tasked with designing a Pachinko game that responds to steel-ball events by moving motors, producing sounds, or displaying animations.

The heart of the game is the ball logger, a 5 channel structure that allows balls to be detected if they pass through any of the individual channels. Each of the channels is lined with copper tape with a break in the middle. When a ball passes through a channel, it bridges the gap in the copper. One side of each channel is attached to the 3.3V pin on an ESP32 and the other side is attached to an analog input GPIO pin with a 10kOhm pull down resistor. The ESP32 is able to read whenever a ball bridges the two sides of a channel.

This ESP32 runs a C-compiled version of RTLola in a dual-core configuration using PlatformIO. Core 0 handles sensing and communication; core 1 runs the RTLola monitor. Core 0 sends sensor events to core 1, and core 1 returns flags when formal specifications are violated or fulfilled. Specifications are written in a `.lola` file, and a build script compiles each specification to a header included by the firmware. As a result, monitor updates are integrated into the normal embedded build process. Students are given a template for this system, so they can focus mainly on the specification itself.

Listing 1 Example RTLola specification for repeated channel events.

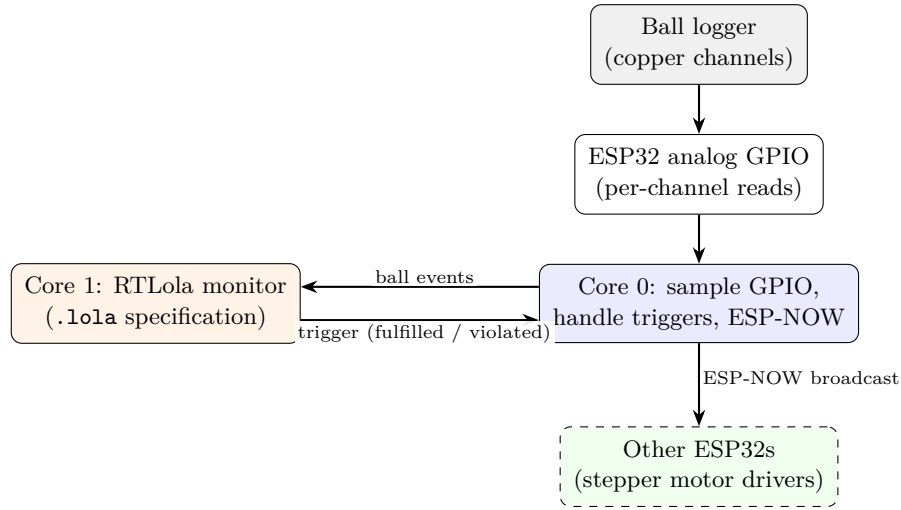
```
input sensor_1: Float64

output ball_1: Bool := sensor_1 > 0.5

output prev_ball_1: Bool := sensor_1.offset(by: -1).defaults(to: 0.0) > 0.5

trigger ball_1 && prev_ball_1 "Channel 1 repeated"
```

In this example, `sensor_1` is sampled from the logger hardware, `ball_1` indicates current-channel occupancy, and `prev_ball_1` captures the prior sample. The trigger fires when occupancy is detected in consecutive samples, which we use as a simple repeated-event pattern before forwarding the corresponding action to the controller logic.



■ **Figure 2** Signal flow on the central (monitor) ESP32: the ball logger is read on core 0, which exchanges events with the RTLola monitor on core 1; triggers return to core 0, which broadcasts actuation commands to distributed motor boards.

107 The students work in groups of 3-5 and are given a blank wooden board, stepper motors
 108 and shaft couplers, 500 brass nails, 200 stainless steel balls, copper tape, the 3D model for
 109 a ball logger, the 3D model for a motor arm, mounting screws for the ball logger and the
 110 motor arm, and a template PlatformIO project file with the RTLola dual core architecture
 111 set up. The template project also included some example formal specifications to get the
 112 students started. The total cost of materials for each board is approximately \$75 USD.

113 Before getting started, the students were asked to submit a work plan detailing the
 114 division of tasks within the group. The main tasks being writing of formal specification and
 115 trigger messages on the “monitor” ESP32, soldering of leads and resistors to the ball logger,
 116 testing stepper motor movements, planning nail locations, testing ESP-NOW communication
 117 between the boards, adding visual artistic elements, and assembly.

118 4 Reflections and Portability

119 Students were able to write formal specifications to control the behavior of their boards
 120 with surprising ease, given the template PlatformIO project. The brunt of the issues and
 121 debugging students ran into was actually on the hardware side. Connecting a distributed
 122 system of microcontrollers together with sensors and motors provided some wiring challenges.
 123 Since runtime monitoring exists in complex embedded systems, this ended up being very
 124 relevant to real world usage.

125 The main limitation of our method to run RTLola on an ESP32 is that RTLola’s C
 126 compiler does not support temporal operators (e.g. `until` and `aggregate`). We could have
 127 opted to create our own runtime monitoring tool specifically for this assignment, but we felt
 128 that using an industry standard tool had more educational value. The students did have a
 129 desire to create temporal aggregates of ball counts, which was not supported by the current
 130 RTLola ESP32 framework. They were, however, able to achieve the same functionality by
 131 aggregating in C code on core 0, and sending the aggregate sums to the monitor. While this
 132 isn’t ideal for teaching aggregation in a formal methods setting, the students were still able
 133 to think in formal specification techniques to achieve the desired behavior of their boards.

We are working with RTLola to have the C-compiler receive temporal logic support in the future.

Given the readily available and low cost physical materials for this project, as well as the ease of setup of the PlatformIO template project and pre-compiled RTLola C-compiler, we feel this assignment is not only portable but customizable. At its core, the assignment is a distributed system of devices and motorized actuators controlled by a central monitor situated on a physical board. The number of distributed devices and motorized actuators and the shape of the physical board are easily variable, and hence so is the context that this assignment can be presented in a classroom. ESP-NOW supports up to 20 devices in unencrypted communication, meaning that this assignment could take the form of a full class collaborative assignment in some cases. Other instantiations of this assignment could include a wide range of other microcontroller sensors and actuators distributed across the board.

References

- 1 Jan Baumeister, Bernd Finkbeiner, Frederik Scheerer, Julian Siber, and Tobias Wagenpfeil. Stream-based monitoring of algorithmic fairness. In *31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, volume 15696 of *Lecture Notes in Computer Science*, pages 60–81. Springer, 2025. URL: <https://arxiv.org/abs/2501.18331>, doi:10.1007/978-3-031-90643-5_4.
- 2 Ben d’Angelo, Sriram Sankaranarayanan, César Sánchez, Will Robinson, Bernd Finkbeiner, Henny B Sipma, Sandeep Mehrotra, and Zohar Manna. Lola: runtime monitoring of synchronous systems. In *12th International Symposium on Temporal Representation and Reasoning (TIME’05)*, pages 166–174. IEEE, 2005.
- 3 Espressif Systems. *ESP-NOW: ESP32 (ESP-IDF Programming Guide)*, 2026. Accessed: 2026-04-30. URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/wifi/esp_now.html.
- 4 Peter Faymonville, Bernd Finkbeiner, Sebastian Schirmer, and Hazem Torfah. A stream-based specification language for network monitoring. In *International Conference on Runtime Verification*, pages 152–168. Springer, 2016.
- 5 Martin Leucker. Teaching runtime verification. In *International Conference on Runtime Verification*, pages 34–48. Springer, 2011.
- 6 Lee Pike, Nis Wegmann, Sebastian Niller, and Alwyn Goodloe. Copilot: monitoring embedded systems. *Innovations in Systems and Software Engineering*, 9(4):235–255, 2013.
- 7 M Todorova and P Armyanov. Runtime verification of computer programs and its application in programming education. In *Global Science and Technology Forum: International Journal of Mathematics, Statistics and Operations Research*, volume 1, pages 105–110, 2012.